Complete Use Case Generation Guide for LLMs

Purpose

This guide teaches you EXACTLY how to generate a complete Salesforce AI Agentic Function use case. Follow every detail precisely to produce production-ready code.

What You Will Generate

When given a use case description, you will generate **EXACTLY 5 FILES + 3 CSV SNIPPET BLOCKS** in a new folder:

1. **[HandlerClassName].apex** — Apex handler class with business logic
2. **[promptname]_PromptCommand.json** — JSON Schema for parameter extraction
3. **AGENT_DESCRIPTION.txt** — Brief agent description for agent record creation
4. **AGENT_SYSTEM_PROMPT.txt** — Complete system prompt defining agent behavior
5. **INTENTS_CONFIG.md** — All intents and actions designed for this agent

Plus, at the end of the output, provide CSV rows ready to append to:

- `Intent_Action_Framework/import-templates/Master_Step1_Intents.csv`
- `Intent_Action_Framework/import-templates/Master_Step2_Intent_Actions.csv`
- `Intent_Action_Framework/import-templates/Master_Step3_Action_Details.csv`

> **Why intents matter:** Intents are the proactive layer of the agent — they detect patterns in user messages and fire automatic actions (creating records, updating fields, running flows, calling Apex) without the user explicitly asking for them. A well-designed agent always has both a reactive layer (skills/functions) and a proactive layer (intents).

Step-by-Step Process

STEP 1: Analyze the Use Case

Given a use case like: "Update case status to 'In Progress' or 'Closed' with resolution notes"

Extract:

1. **Primary Purpose**: What is the main function? (Update case status)
2. **Salesforce Object**: Which object? (Case)
3. **Operation Type**: CREATE, READ, UPDATE, DELETE (UPDATE)
4. **Required Parameters**: What MUST be provided? (caseId, status)
5. **Optional Parameters**: What is optional? (resolutionNotes)
6. **Business Logic**: Any special rules? (Validate status, prevent reopening closed cases)

STEP 2: Define Naming Convention

Follow this pattern EXACTLY:

Component	Pattern	Example
AI Prompt Name	`operation_Object_by_Identifier`	`update_Case_Status`

Component	Pattern	Example
Handler Class	`OperationObjectAgenticHandler`	`UpdateCaseStatusAgenticHandler`
Request Param	Same as AI Prompt Name	`update_Case_Status`
Folder Name	Capitalize words with underscores	`Update_Case_Status`

Rules:

- Use camelCase for handler class names
- Use snake_case for prompt names (e.g. `create_Opportunity`, `find_Account_by_Name` – note: Object name capitalized)
- Use PascalCase_With_Underscores for folder names
- Operation verbs: create, find, update, delete, search, calculate
- Always include the object name

⚠ **CRITICAL – Prompt Name Case Sensitivity:**

- AI Prompt **Name** is **case-sensitive**. `create_Opportunity` (correct) vs `create_opportunity` (wrong) are different records.
- Use the canonical form: `operation_Object` – e.g. `create_Opportunity`, `update_Case`, `find_Account_by_Name`.
- **Never create duplicate prompts** with different casing (e.g. `create_opportunity` and `create_Opportunity`). The wrong one may be invoked, causing "Method is not defined" or the operation to fail silently. When an operation does not work, check for duplicate/similar prompt names and remove the incorrect one.

STEP 3: Create Folder Structure

Create folder: `use-cases/[Use_Case_Name]/`

Example: `use-cases/Update_Case_Status/`

FILE 1: Apex Handler Class

Template Structure

```
/**
 * @description
 *   [ClassName] implements the AIAgenticInterface and serves as a handler
 *   for [purpose]. [What it does in detail].
 *
 * @author              : AI Agentic Architecture
 * @group               : Plumcloud Labs
 * @last modified on    : [MM-DD-YYYY]
 */
public with sharing class [HandlerClassName] implements AIAgenticInterface {

    // SECTION 1: Permission Check Method
    private Boolean hasObjectPerm(String sObjectName, String permType) {
        Schema.DescribeSObjectResult describeResult =
Schema.getGlobalDescribe().get(sObjectName).getDescribe();
```

```
        if (permType == 'read')      return describeResult.isAccessible();
        if (permType == 'create')    return describeResult.isCreateable();
        if (permType == 'update')    return describeResult.isUpdateable();
        if (permType == 'delete')    return describeResult.isDeletable();
        return false;
    }

    // SECTION 2: Error Response Methods
    private String errorResponse(Exception ex) {
        return JSON.serialize(new Map<String, Object>{
            'success' => false,
            'status' => 'errored',
            'message' => ex.getMessage(),
            'stackTrace' => ex.getStackTraceString()
        });
    }

    private String errorResponse(String message) {
        return JSON.serialize(new Map<String, Object>{
            'success' => false,
            'status' => 'errored',
            'message' => message
        });
    }

    // SECTION 3: Request Dispatcher
    public String executeMethod(String requestParam, Map<String, Object> parameters) {
        try {
            switch on requestParam {
                when '[request_param_name]' {
                    return [methodName](parameters);
                }
                when else {
                    return errorResponse('Method is not defined. This handler only
supports: [request_param_name]');
                }
            }
        } catch (Exception ex) {
            return errorResponse(ex);
        }
    }

    // SECTION 4: Implementation Method
    public String [methodName](Map<String, Object> parameters) {
        // 1. Check permissions
        if (!hasObjectPerm('[ObjectName]', '[operation]')) {
            return errorResponse('Insufficient [operation] permission on [ObjectName]
object.');
        }

        // 2. Validate required parameters
        if (!parameters.containsKey('requiredField') ||
            String.isBlank(String.valueOf(parameters.get('requiredField')))) {
            return errorResponse('requiredField is required.');
        }

        // 3. Extract and validate parameters
```

```
        String field1 = String.valueOf(parameters.get('field1'));

        // Additional validation (e.g., ID format, enum values)
        // ...

        // 4. Query if needed (for UPDATE/DELETE operations)
        List<[SObject]> records = [
            SELECT Id, Field1, Field2
            FROM [SObject]
            WHERE Id = :recordId
            LIMIT 1
        ];

        if (records.isEmpty()) {
            return errorResponse('[SObject] not found with ID: ' + recordId);
        }

        [SObject] record = records[0];

        // 5. Perform DML operation
        try {
            // For CREATE:
            [SObject] newRecord = new [SObject]();
            newRecord.Field1 = field1;
            insert newRecord;

            // For UPDATE:
            record.Field1 = field1;
            update record;

            // For DELETE:
            delete record;

            // For QUERY/READ:
            // Return query results

            // 6. Build redirect URL (for CREATE/UPDATE)
            String redirectUrl = URL.getOrgDomainUrl().toExternalForm() + '/' +
record.Id;

            // 7. Return success response
            return JSON.serialize(new Map<String, Object>{
                'success' => true,
                'status' => 'success',
                'message' => 'Successfully [action] [object]',
                'recordId' => record.Id,
                '[otherRelevantFields]' => 'values',
                'redirectUrl' => redirectUrl,
                'action' => 'redirect'
            });

        } catch (DmlException dmlEx) {
            return JSON.serialize(new Map<String, Object>{
                'success' => false,
                'status' => 'errored',
                'message' => 'DML Error: ' + dmlEx.getDmlMessage(0)
            });
```

```
        } catch (Exception ex) {
            return errorResponse(ex);
        }
    }

    // SECTION 5: Helper Methods (if needed)
    private Boolean isValidId(String recordId) {
        if (String.isBlank(recordId)) {
            return false;
        }
        Integer idLength = recordId.length();
        if (idLength != 15 && idLength != 18) {
            return false;
        }
        return recordId.isAlphanumeric();
    }
}
```

Critical Requirements for Apex Class:

1. **MUST use**: `public with sharing class`
2. **MUST implement**: `AIAgenticInterface`
3. **MUST have**: `hasObjectPerm()` method (exact signature)
4. **MUST have**: Two `errorResponse()` methods (one for Exception, one for String)
5. **MUST have**: `executeMethod()` with switch statement
6. **MUST validate**: All required parameters before DML
7. **MUST check**: Object-level permissions before any operation
8. **MUST handle**: DmlException separately from general Exception
9. **MUST return**: Standardized JSON response format
10. **MUST include**: Redirect URL for CREATE and UPDATE operations

Response Format Standards:

Success Response (CREATE/UPDATE):

```
{
  "success": true,
  "status": "success",
  "message": "Successfully [action] [object]",
  "recordId": "ID",
  "[recordIdentifier]": "Value (e.g., CaseNumber, Name)",
  "[relevantFields]": "values",
  "redirectUrl": "URL",
  "action": "redirect"
}
```

Success Response (READ/QUERY):

```
{
  "success": true,
  "status": "success",
```

```
  "message": "Found [X] records",
  "[dataArray]": [...],
  "count": X,
  "[calculatedFields]": "values"
}
```

Error Response:

```
{
  "success": false,
  "status": "errored",
  "message": "Error description"
}
```

Validation Patterns:

For ID validation:

```
if (!isValidId(recordId) || !recordId.startsWith('[prefix]')) {
    return errorResponse('Invalid [Object] ID format. Must be 15 or 18 characters
starting with [prefix].');
}
```

For enum validation:

```
List<String> validValues = new List<String>{'Value1', 'Value2', 'Value3'};
if (!validValues.contains(fieldValue)) {
    return errorResponse('Invalid value. Valid values: ' + String.join(validValues, ',
'));
}
```

For required field validation:

```
if (!parameters.containsKey('fieldName') ||
    String.isBlank(String.valueOf(parameters.get('fieldName')))) {
    return errorResponse('fieldName is required.');
}
```

FILE 2: Prompt Command JSON Schema

Template Structure

```
{
  "type": "object",
```

```
    "required": ["field1", "field2"],
    "properties": {
      "fieldName": {
        "type": "string|number|boolean",
        "maxLength": 255,
        "minimum": 0,
        "maximum": 100,
        "enum": ["value1", "value2"],
        "description": "ONLY the [field] portion. Extract from '[Pattern]: [value]' or
'[Pattern] [value]' patterns. Example: if input is '[Example Input]', extract '[Example
Output]' only. This field is REQUIRED/OPTIONAL."
      }
    }
  }
```

CRITICAL RULES FOR DESCRIPTIONS:

Every field description MUST follow this EXACT format:

1. **Start with**: "ONLY the [field] portion"
2. **Show patterns**: "Extract from '[Pattern]: [value]' or '[Alternative Pattern]' patterns"
3. **Give example**: "Example: if input is '[Input]', extract '[Output]' only"
4. **State requirement**: "This field is REQUIRED" or "This field is OPTIONAL"
5. **Add default if applicable**: "Default: [value] if not specified"

Examples by Field Type:

String Field (ID):

```
"caseId": {
  "type": "string",
  "maxLength": 18,
  "description": "ONLY the Case ID (Salesforce 15 or 18 character ID). Extract from
'Case ID: [value]' or 'Case: [value]' patterns. Example: if input is 'Update case
500xx000000ABCD', extract '500xx000000ABCD' only. Case IDs start with '500'. This field
is REQUIRED."
}
```

String Field (Enum):

```
"status": {
  "type": "string",
  "enum": ["New", "Working", "In Progress"],
  "description": "ONLY the status value. Extract from 'Status: [value]' or 'to [value]'
or 'mark as [value]' patterns. Example: if input is 'Update to In Progress', extract
'In Progress' only. Valid values: 'New', 'Working', 'In Progress'. This field is
REQUIRED."
}
```

Number Field:

```
"minimumProbability": {
  "type": "number",
  "minimum": 0,
  "maximum": 100,
  "description": "ONLY the probability percentage value. Extract from 'probability:
[value]' or 'probability over [value]%' patterns. Example: if input is 'probability
over 70%', extract 70 only. Default: 70 if not specified. This field is OPTIONAL."
}
```

Boolean Field:

```
"includeClosed": {
  "type": "boolean",
  "description": "ONLY the boolean value. Extract from 'include closed' or 'with
closed' patterns. Example: if input is 'show opportunities including closed', extract
true. Default: false. This field is OPTIONAL."
}
```

Text Field:

```
"resolutionNotes": {
  "type": "string",
  "description": "ONLY the resolution notes or comments. Extract from 'Resolution:
[value]' or 'Notes: [value]' patterns. Example: if input is 'Close with resolution:
Fixed network issue', extract 'Fixed network issue' only. This field is OPTIONAL."
}
```

Validation Constraints to Include:

- **maxLength**: For string fields (typically 255 for short text, 32000 for long text)
- **minimum/maximum**: For number fields
- **enum**: For picklist/status fields (list ALL valid values)
- **required array**: List all required field names

FILE 3: AGENT_DESCRIPTION.txt

Format:

Simple plain text, 1-3 sentences describing what the agent does for end users.

Template:

```
Create a [agent type] agent that can [list capabilities]. The agent should be
[personality traits] and capable of [key behaviors].
```

Examples:

Update Case Status:

```
Create a case management agent that can update case statuses to various states
including In Progress, Escalated, On Hold, and Closed, with the ability to add
resolution notes when closing cases. The agent should be efficient, detail-oriented,
and ensure proper case status tracking throughout the support lifecycle.
```

Find Opportunities:

```
Create an opportunity intelligence agent that can help sales teams find high-
probability opportunities in their pipeline, analyze deal confidence levels, calculate
total pipeline value for forecasting, and identify which deals are most likely to
close. The agent should be professional, data-driven, and capable of providing
actionable insights and recommendations for deal prioritization.
```

Guidelines:

- Focus on WHAT the agent does for users (not technical implementation)
- Include personality traits (professional, efficient, friendly, data-driven)
- Mention key capabilities
- Keep it business-focused and user-friendly
- 50-150 words maximum

FILE 4: AGENT_SYSTEM_PROMPT.txt

Structure:

The system prompt defines HOW the agent behaves. Use plain text format (NO HTML tags).

Template:

```
You are a [Agent Type] Agent responsible for [primary responsibility].

CORE RESPONSIBILITIES:
- [Responsibility 1]
- [Responsibility 2]
- [Responsibility 3]

COMMUNICATION STYLE:
Your communication should be [style traits], using [type] language appropriate for
[audience].

[CRITICAL SECTIONS BASED ON USE CASE - see examples below]

LIMITATIONS:

You CANNOT:
- [Limitation 1]
- [Limitation 2]
```

```
- [Limitation 3]

If a user requests an action outside your capabilities, politely direct them to the
appropriate agent or suggest they perform the action manually in Salesforce.
```

Critical Sections Based on Operation Type:

For UPDATE/CREATE Operations (requires user input):

Add this section:

```
CRITICAL: [ALWAYS/NEVER RULES]

NEVER claim to have [action] unless you have successfully called the [function_name]
function.

ALWAYS follow this process:
1. Request [Required Info]: If user doesn't provide [info], ask for it
2. Confirm [Details]: Clarify any ambiguous details
3. Execute Function: Call [function_name] with parameters
4. Report Result: Only after success, confirm with details from response

RESPONSE FORMAT BASED ON FUNCTION RESULTS:

ONLY respond with success messages if the function call returns success: true.

- On Success: "[Success message with actual data from response]"
- On Failure: "Unable to [action]: [error message]. Please check [input] and try
again."
- Missing Required Info: "I need [info] to [action]. Please provide [details]."
```

For READ/QUERY Operations:

Add this section:

```
WHEN PRESENTING RESULTS, FOLLOW THIS FORMAT:

1. Summary: "[X] [objects] found with [criteria]"
2. Key Metrics: "[Important numbers/totals]"
3. Top Items: List [3-5] most relevant items
4. Insights: Brief analysis of what the data means
5. Next Steps: Suggest actionable recommendations

KEY METRICS TO ALWAYS INCLUDE:
- [Metric 1]
- [Metric 2]
- [Metric 3]
```

Example Interactions:

Always include 2-3 example interactions showing:

- What user says
- What agent does (function call)
- What agent responds (based on success/failure)

Format:

```
EXAMPLE INTERACTIONS:

Example 1:
User: "[User input]"
You: [Call function_name with parameters]
If Success: "[Response using actual data from function]"

Example 2:
User: "[User input without required info]"
You: "[Ask for required information]"
```

Full Examples:

For UPDATE operations:

```
You are a Case Management Agent responsible for updating case statuses and adding
resolution notes.

CORE RESPONSIBILITIES:
- Update case statuses to 'New', 'Working', 'In Progress', 'Escalated', 'On Hold',
'Closed', 'Closed - Resolved', and 'Closed - Not Resolved'
- Add detailed resolution notes when closing cases to document the solution provided
- Ensure proper tracking of case statuses throughout the support lifecycle
- Validate case IDs before making any updates

COMMUNICATION STYLE:
Your communication should be efficient and detail-oriented, using professional language
appropriate for case management.

CRITICAL: ALWAYS ASK FOR CASE ID FIRST

NEVER claim to have updated a case unless you have successfully called the
update_Case_Status function with a valid case ID.

ALWAYS follow this process:
1. Request Case ID: If the user doesn't provide a case ID, ask for it
2. Confirm Status: If the user doesn't specify the exact status, clarify options
3. Execute Function: Call update_Case_Status with the case ID and status
4. Report Result: Only after the function succeeds, confirm the action with details
from the response

RESPONSE FORMAT BASED ON FUNCTION RESULTS:

ONLY respond with success messages if the function call returns success: true.
```

```
- On Success: "Case [CaseNumber] has been successfully updated from '[OldStatus]' to
'[NewStatus]'. View Case: [redirectUrl]"
- On Failure: "Unable to update case: [error message]. Please check the Case ID and try
again."
- Missing Case ID: "I need the Case ID to update the status. Please provide the 15 or
18 character Case ID (starts with '500')."

IMPORTANT GUIDELINES:
- Never fake responses: Do NOT say "the case has been updated" unless the function
returned success
- Always validate inputs: Confirm you have a valid Case ID before attempting any action
- Be transparent: If you don't have enough information, ask for it
- Use actual data: Use the case number, old status, and new status from the function
response

EXAMPLE INTERACTIONS:

Example 1:
User: "Can you close this case"
You: "I can help you close a case. Please provide the Case ID (a 15 or 18 character ID
starting with '500')."

Example 2:
User: "Close case 500xx000000ABCD"
You: [Call update_Case_Status with caseId: "500xx000000ABCD", status: "Closed"]
If Success: "Case 00001234 has been successfully updated from 'In Progress' to
'Closed'. View Case: [URL]"

LIMITATIONS:

You CANNOT:
- Create new cases (use the case creation agent for that)
- Modify case details such as subject, description, priority
- Reopen cases that have been closed
- Delete or merge cases
- Update a case without a Case ID

If a user requests an action outside your capabilities, politely direct them to the
appropriate agent or suggest they perform the action manually in Salesforce.

REMEMBER: NEVER claim an action was completed unless you actually called the function
and it returned success: true.
```

For READ operations:

```
You are an Opportunity Intelligence Agent responsible for helping sales teams find and
analyze high-probability opportunities.

CORE RESPONSIBILITIES:
- Find opportunities based on probability thresholds (default: 70% or higher)
- Calculate total pipeline value for filtered opportunities
- Provide data-driven insights on deal confidence levels and pipeline health
- Identify which deals are most likely to close for prioritization

COMMUNICATION STYLE:
```

```
Your communication should be professional and data-driven, using clear business
language appropriate for sales teams.

WHEN PRESENTING OPPORTUNITY RESULTS, FOLLOW THIS FORMAT:

1. Summary: "Found [X] opportunities with [Y]% or higher probability."
2. Total Pipeline Value: "Total potential revenue: [Amount] [Currency]."
3. Top Opportunities: List 3-5 highest value or probability deals.
4. Insights: Provide brief analysis of what the data indicates about pipeline health.
5. Next Steps: Suggest actionable recommendations for the sales team.

KEY METRICS TO ALWAYS INCLUDE:
- Number of opportunities found
- Total pipeline value (sum of all amounts)
- Average deal size (if multiple opportunities)
- Probability range used for filtering
- Direct links to view opportunities in Salesforce

PROACTIVE RECOMMENDATIONS:

Based on the data you retrieve, provide actionable insights such as:
- Which deals to prioritize for immediate attention
- Pipeline health assessment (strong, needs attention, at risk)
- Suggestions for forecast adjustments

EXAMPLE RESPONSE:

Query: "Show me our best opportunities"

Your Response:

OPPORTUNITY INTELLIGENCE REPORT

Found 8 opportunities with 70% or higher probability

Total Pipeline Value: €1,135,000

TOP OPPORTUNITIES:
- GenePoint - €85,000 (90% probability) - Closes Dec 15, 2025
- Burlington Textiles - €50,000 (80% probability) - Closes Jan 10, 2026

INSIGHTS:
Your high-confidence pipeline is strong with €1.1M in likely revenue. Focus on the 90%
deal for Q4 close.

RECOMMENDED NEXT STEPS:
- Review the GenePoint deal for final blockers
- Schedule close calls for Q4 opportunities

Would you like to adjust the probability threshold?

LIMITATIONS:

You CANNOT:
- Modify opportunity records
- Create new opportunities
- Change probability values or stages
```

```
- Predict future probability changes (only report current values)

Always ensure you are providing accurate, current data from Salesforce. The data is
retrieved at the moment of the query.
```

FILE 5: INTENTS_CONFIG.md

This file documents every intent designed for the agent. It is the single source of truth for what gets imported into the Intent Action Framework via Data Loader or Anonymous Apex.

Why every agent needs intents

Functions (skills) are triggered by explicit user commands: "update this opportunity". Intents fire automatically when the AI detects a specific pattern in the conversation — the user doesn't have to ask for the action directly. Together they make the agent both reactive and proactive.

Step A — Intent Count and Naming Rules

Design **4 to 8 intents** per agent. More is fine for complex agents (e.g. SP Energy Bot has 21). Fewer than 4 is usually a sign that the agent needs more coverage.

Naming rules:

Agent type	Intent name format	Example
Internal / CRM agents	`snake_case`	`deal_stalled`, `account_at_risk`
Consumer-facing / chatbots	`#Keyword#` hashtag format	`#Emergency#`, `#Mental Health#`
Both formats	Pick one and stay consistent within an agent	—

Standard intents that every agent should have:

Intent	Purpose
`[agent]_greeting`	Welcome message when user opens chat
`[agent]_out_of_scope`	Graceful rejection when user asks for something outside capabilities

Domain intents — designed around the agent's specific capabilities (see Step B).

Step B — Designing domain intents from the agent's system prompt

Read the system prompt and ask: "*What situations can arise in a conversation with this agent that would benefit from automatic backend action?*" Common triggers:

Signal in conversation	Intent to design
User reports a bad outcome (loss, failure, crisis)	Intent to create a Case or Task

Signal in conversation	Intent to design
User expresses frustration, urgency, or risk	Intent to escalate and create a record
User reaches a milestone (deal won, case resolved)	Intent to celebrate and trigger follow-up
User asks for something the agent cannot do	Intent to redirect gracefully
User is confused about options, steps, or values	Intent to provide structured canned guidance
User signals data needs updating (stalled, overdue, flagged)	Intent to update a field
Situation involves complex multi-step backend logic	Intent to invoke a Flow or Apex class

---

Step C — Choosing action types for each intent

Every intent must have **at least one Canned Response** (immediate user-facing feedback). Then add backend actions based on severity and complexity:

Action Type	When to use
Canned Response	Always — every intent must have one. Provides immediate, structured feedback. Set per language if multilingual.
Create Record (Task)	When the intent signals something a human team member needs to follow up on. Subject and Priority should reflect urgency.
Create Record (Case)	When the intent signals a customer support issue that needs formal tracking and assignment.
Update Field	When the conversation reveals a meaningful state change that should be persisted on the related record (e.g. flag an account as at-risk, note a deal as stalled). Usually prepend a status prefix to the Description field.
Flow	When multiple backend steps need to run in sequence, or when you need to notify multiple teams, create records with dependencies, or orchestrate processes too complex for a single record operation.
Apex	When you need: a custom real-time response message, chatter posts, external system calls, conditional business logic, or anything that requires code-level intelligence rather than declarative config.

Sequence order: Run backend record operations first (Create Record, Update Field, Flow, Apex), then Canned Response last. This ensures data is saved before the user sees the response.

Step D — INTENTS_CONFIG.md template

```
# [Agent Name] — Intents Configuration

**Agent:** [Agent Name]
**Total Intents:** [N]
```

```
**CSV Prefix:** [PREFIX] (e.g. CRM, OPP, CPQ, DOC, PIPE, CASE, SPE)

---

## Intent Design Summary

| Ext_ID | Name | Seq | Action Types | Notes |
|--------|------|-----|-------------|-------|
| [PREFIX]-INT-001 | [intent_name] | 1 | Canned Response | Greeting |
| [PREFIX]-INT-002 | [intent_name] | 2 | Canned Response, Create Record (Task), Update
Field | At-risk signal |
| ... | | | | |

---

## Intent Definitions

### [PREFIX]-INT-001 — [intent_name]

**Trigger description (set in GPTfy UI → Intent → Description):**
> [Exact trigger phrases and scenarios. Be specific about the user words/patterns that
should fire this intent.]

**Sequence:** [N]
**Is Active:** true

#### Actions

**Action 1 — Canned Response (Seq 1)**
- Language: English
- Text: "[Full canned response text to show user]"

**Action 2 — Create Record: Task (Seq 2)**
- Object: Task
- Field mappings:
  | Field | Type | Value / AI Instruction |
  |-------|------|----------------------|
  | Subject | Hardcoded | [Exact subject text] |
  | Description | AI Extracted | [Instruction for AI: what to extract from
conversation] |
  | Priority | Hardcoded | High / Normal / Low |
  | Status | Hardcoded | Not Started |

**Action 3 — Update Field (Seq 3)** *(if applicable)*
- Object: [ObjectAPIName]
- Field: Description
- Type: AI Extracted
- Instruction: [What the AI should write into the field, including any prefix like
STALLED: or AT-RISK:]

**Action 4 — Flow (Seq 4)** *(if applicable)*
- Flow API Name: [FlowAPIName]
- Purpose: [What the flow does]

**Action 5 — Apex (Seq 5)** *(if applicable)*
- Apex Class Name: [ClassName]
- Return Type: Replace Message / Append to Message
```

```
- Class location: `force-app/main/default/classes/[ClassName].cls`
- Purpose: [What the class does]

---

[Repeat for each intent]

---

## Apex Classes Required

| Class Name | Intent(s) | Purpose |
|------------|-----------|---------|
| [ClassName] | [INT-00N] | [What it does] |

## Flows Required

| Flow API Name | Intent(s) | Purpose |
|---------------|-----------|---------|
| [FlowAPIName] | [INT-00N] | [What it does] |
```

Step E — Generating the 3 CSV snippet blocks

After writing INTENTS_CONFIG.md, produce three clearly labelled CSV snippet blocks. These rows are appended to the master files in `Intent_Action_Framework/import-templates/`.

Block 1 — Append to `Master_Step1_Intents.csv`

Use this column order (no header row in the snippet — the file already has one):

```
Ext_ID,Name,ccai_qa__AI_Agent__r:Name,_Sequence,_Is_Active,_Description
```

One row per intent. The `_Description` value is the trigger description — copy it exactly from the INTENTS_CONFIG.md intent definition.

Block 2 — Append to `Master_Step2_Intent_Actions.csv`

Use this column order:

```
Ext_ID,Intent_Ext_ID,ccai_qa__AI_Agent_Intent__r:Name,_Seq,_Action_Type,_Language,_Cann
ed_Response_Text,_Object_API_Name,_Flow_API_Name,_Apex_Class_Name,_Apex_Return_Type
```

One row per action per intent. Leave empty cells for columns not applicable to the action type.

Block 3 — Append to `Master_Step3_Action_Details.csv`

Use this column order:

```
Ext_ID,Action_Ext_ID,ccai_qa__AI_Intent_Action__c,ccai_qa__Field_API_Name__c,ccai_qa__T
ype__c,_Hardcoded_Value_Or_AI_Instruction
```

One row per field mapping. The `ccai_qa__AI_Intent_Action__c` column is always blank — it is filled after Data Loader Step 2 export.

Step F — Ext_ID prefix convention

Derive a short 2–6 letter prefix from the agent name:

Agent name	Prefix	Example Ext_IDs
Salesforce CRM Agent	CRM	CRM-INT-001, CRM-INT-001-ACT-001
Update Opportunity Stage CloseDate	OPP	OPP-INT-001
CPQ Agent	CPQ	CPQ-INT-001
Document Intelligence Agent	DOC	DOC-INT-001
Find Opportunity by Probability	PIPE	PIPE-INT-001
Update Case Status	CASE	CASE-INT-001
SP Energy Bot	SPE	SPE-INT-001
[New agent]	[3-4 letters from name]	[PREFIX]-INT-001

Action Ext_IDs extend the intent Ext_ID: `CRM-INT-002-ACT-003` Detail Ext_IDs extend the action Ext_ID: `CRM-INT-002-ACT-003-DTL-002`

Quality Checklist

Before outputting files, verify:

Apex Handler Class:

- ☑ Uses `public with sharing class`
- ☑ Implements `AIAgenticInterface`
- ☑ Has `hasObjectPerm()` method
- ☑ Has two `errorResponse()` methods
- ☑ Has `executeMethod()` with switch statement
- ☑ Validates all required parameters
- ☑ Checks object-level permissions
- ☑ Handles DmlException separately
- ☑ Returns standardized JSON response
- ☑ Includes redirect URL (for CREATE/UPDATE)

JSON Schema:

- ☑ All required fields in `required` array
- ☑ Descriptions start with "ONLY the [field] portion"

- ☑ Shows extraction patterns
- ☑ Includes concrete examples
- ☑ States REQUIRED or OPTIONAL
- ☑ Has validation constraints (maxLength, enum, min/max)
- ☑ For enum fields, lists ALL valid values

Agent Description:

- ☑ 1-3 sentences
- ☑ User-focused (not technical)
- ☑ Includes personality traits
- ☑ Business-friendly language

Agent System Prompt:

- ☑ Plain text format (no HTML)
- ☑ Defines core responsibilities
- ☑ Specifies communication style
- ☑ Includes critical behavioral rules
- ☑ Has response format guidelines
- ☑ Provides example interactions
- ☑ Lists limitations clearly
- ☑ For UPDATE/CREATE: Emphasizes NOT to fake responses
- ☑ For READ: Includes data presentation format

INTENTS_CONFIG.md:

- ☑ Has a greeting intent
- ☑ Has an out-of-scope / redirect intent
- ☑ Has 4–8+ domain intents reflecting the agent's actual capabilities
- ☑ Every intent has at least one Canned Response action
- ☑ Backend actions (Create Record, Update Field, Flow, Apex) are included where appropriate
- ☑ All 5 action types are used at least once across the full intent set (where scenarios justify them)
- ☑ Apex intents have a corresponding `.cls` file path documented
- ☑ Flow intents have a Flow API name documented
- ☑ Trigger descriptions are specific enough to reliably fire (not too broad, not too narrow)
- ☑ Sequence numbers are assigned (backend actions first, Canned Response last)
- ☑ Field mappings for Create Record / Update Field intents list all fields with correct Type

CSV Snippet Blocks:

- ☑ Block 1 (Step 1) has one row per intent with correct Ext_ID prefix
- ☑ Block 2 (Step 2) has one row per action with `ccai_qa__AI_Agent_Intent__r:Name` matching the intent Name
- ☑ Block 3 (Step 3) has one row per field mapping; `ccai_qa__AI_Intent_Action__c` column left blank
- ☑ All `_NOTE_` / underscore columns are present and populated with config instructions
- ☑ No header row included in the snippet (master files already have headers)

Common Patterns by Operation Type

CREATE Operations:

- Check `create` permission
- Validate all required fields
- Create new SObject instance
- Set all fields from parameters
- `insert record;`
- Return recordId, recordName, redirectUrl

READ/QUERY Operations:

- Check `read` permission
- Build dynamic SOQL query
- Filter based on parameters
- Calculate aggregates if needed
- Return array of records + metadata (count, totals)

UPDATE Operations:

- Check `read` and `update` permissions
- Query existing record first
- Validate record exists
- Update fields from parameters
- `update record;`
- Return old and new values, redirectUrl

DELETE Operations:

- Check `delete` permission
- Query existing record first
- Validate record exists
- Confirm deletion not prevented by business rules
- `delete record;`
- Return confirmation message

Multi-Operation Handler Pattern

When to Use Multi-Operation Handlers

Use a **single handler class with multiple operations** when:

- Operations are related to the same primary object (e.g., Account)
- Operations share common business logic or validation
- Operations provide complementary functionality (e.g., find Account + find related Objects)

Example: Account Intelligence Handler

- `find_Account_by_Name` - Find account by name
- `find_Contacts_for_Account` - Get contacts for an account
- `find_Opportunities_for_Account` - Get opportunities for an account
- `find_Cases_for_Account` - Get cases for an account

Multi-Operation Handler Structure

```
public with sharing class AccountIntelligenceHandler implements AIAgenticInterface {

    // SECTION 1: Permission Check Method (same as single-operation)
    private Boolean hasObjectPerm(String sObjectName, String permType) {
        // ... standard implementation
    }

    // SECTION 2: Error Response Methods (same as single-operation)
    private String errorResponse(Exception ex) {
        // ... standard implementation
    }

    private String errorResponse(String message) {
        // ... standard implementation
    }

    // SECTION 3: Request Dispatcher - Multiple Operations
    public String executeMethod(String requestParam, Map<String, Object> parameters) {
        try {
            switch on requestParam {
                when 'find_Account_by_Name' {
                    return findAccountByName(parameters);
                }
                when 'find_Contacts_for_Account' {
                    return findContactsForAccount(parameters);
                }
                when 'find_Opportunities_for_Account' {
                    return findOpportunitiesForAccount(parameters);
                }
                when 'find_Cases_for_Account' {
                    return findCasesForAccount(parameters);
                }
                when else {
                    return errorResponse('Method is not defined. Supported methods:
find_Account_by_Name, find_Contacts_for_Account, find_Opportunities_for_Account,
find_Cases_for_Account');
                }
            }
        } catch (Exception ex) {
            return errorResponse(ex);
        }
    }

    // SECTION 4: Implementation Methods - One per Operation
    public String findAccountByName(Map<String, Object> parameters) {
        // Implementation for finding account by name
    }

    public String findContactsForAccount(Map<String, Object> parameters) {
        // Implementation for finding related contacts
    }

    public String findOpportunitiesForAccount(Map<String, Object> parameters) {
        // Implementation for finding related opportunities
    }
```

```
    public String findCasesForAccount(Map<String, Object> parameters) {
        // Implementation for finding related cases
    }

    // SECTION 5: Shared Helper Methods
    private Boolean isValidId(String recordId) {
        // Shared validation logic
    }
}
```

Multi-Operation Prompt Setup

For a multi-operation handler, you create **multiple AI Prompt records** in Salesforce:

AI Prompt Record	Request_Param__c	Handler_Class_Name__c	Prompt_Command__c
Find Account by Name	`find_Account_by_Name`	`AccountIntelligenceHandler`	JSON schema for account search
Find Contacts for Account	`find_Contacts_for_Account`	`AccountIntelligenceHandler`	JSON schema for contact query
Find Opportunities for Account	`find_Opportunities_for_Account`	`AccountIntelligenceHandler`	JSON schema for opportunity query
Find Cases for Account	`find_Cases_for_Account`	`AccountIntelligenceHandler`	JSON schema for case query

Key Points:

- Same `Handler_Class_Name__c` for all prompts
- Different `Request_Param__c` for each operation
- Different `Prompt_Command__c` (JSON schema) for each operation
- Each prompt can have its own agent or share one agent

Fuzzy Search Pattern for Name-Based Lookups

When searching for records by name (Account, Contact, etc.), implement fuzzy matching to handle typos and misspellings:

Problem: User types "gaslume" but actual record is "Galume Energy"

- Simple SOQL LIKE won't find it
- User gets frustrated with "not found" errors

Solution: Multi-strategy search with SOSL + SOQL fallback

```
/**
 * @description
 *   Performs fuzzy account search using SOSL (for typo tolerance) with fallback to
SOQL LIKE.
 *   Handles misspellings like "gaslume" finding "Galume Energy".
```

```
 */
private List<Account> fuzzySearchAccounts(String searchTerm, Integer resultLimit) {
    List<Account> accounts = new List<Account>();

    try {
        // Strategy 1: Use SOSL for fuzzy matching (handles typos)
        String soslSearchTerm = searchTerm + '*';
        List<List<SObject>> searchResults = [
            FIND :soslSearchTerm IN NAME FIELDS
            RETURNING Account(
                Id, Name, Type, Industry, AnnualRevenue,
                Phone, Website, BillingCity, BillingState,
                BillingCountry, Owner.Name, CreatedDate
                ORDER BY Name
                LIMIT :resultLimit
            )
        ];

        if (!searchResults.isEmpty() && !searchResults[0].isEmpty()) {
            accounts = (List<Account>) searchResults[0];
        }
    } catch (Exception soslEx) {
        // SOSL failed, will try SOQL fallback
        System.debug('SOSL search failed: ' + soslEx.getMessage());
    }

    // Strategy 2: Fallback to SOQL LIKE if SOSL returned no results
    if (accounts.isEmpty()) {
        try {
            String searchPattern = '%' + searchTerm + '%';
            accounts = [
                SELECT Id, Name, Type, Industry, AnnualRevenue,
                       Phone, Website, BillingCity, BillingState,
                       BillingCountry, Owner.Name, CreatedDate
                FROM Account
                WHERE Name LIKE :searchPattern
                ORDER BY Name
                LIMIT :resultLimit
            ];
        } catch (Exception soqlEx) {
            System.debug('SOQL fallback failed: ' + soqlEx.getMessage());
        }
    }

    return accounts;
}
```

Usage in methods:

```
// In findAccountByName
List<Account> accounts = fuzzySearchAccounts(accountName, resultLimit);

// In resolveAccountId helper
List<Account> accounts = fuzzySearchAccounts(accountName, 1);
```

Why this works:

- **SOSL** uses Salesforce's search engine with built-in fuzzy matching
- Handles typos, phonetic similarities, common misspellings
- **SOQL LIKE** fallback ensures partial matches still work
- Example: "gaslume" → SOSL finds "Galume Energy"
- Example: "lume" → SOQL finds any account with "lume" in name

Best Practice: Use fuzzy search for all user-facing name lookups (Accounts, Contacts, Leads, etc.)

Related Object Query Patterns

When querying related objects, follow these patterns:

Pattern 1: Direct Relationship Query (when you have the parent ID)

```
// User provides accountId directly
if (!parameters.containsKey('accountId') ||
    String.isBlank(String.valueOf(parameters.get('accountId')))) {
    return errorResponse('accountId is required.');
}

String accountId = String.valueOf(parameters.get('accountId'));

List<Contact> contacts = [
    SELECT Id, Name, Email, Phone, Title
    FROM Contact
    WHERE AccountId = :accountId
    ORDER BY Name
];
```

Pattern 2: Find Parent First, Then Related Objects

```
// User provides account name, find account first
String accountName = String.valueOf(parameters.get('accountName'));

List<Account> accounts = [
    SELECT Id, Name
    FROM Account
    WHERE Name LIKE :('%' + accountName + '%')
    LIMIT 1
];

if (accounts.isEmpty()) {
    return errorResponse('Account not found with name: ' + accountName);
}

// Now query related objects
List<Contact> contacts = [
    SELECT Id, Name, Email, Phone, Title
    FROM Contact
```

```
    WHERE AccountId = :accounts[0].Id
    ORDER BY Name
];
```

Pattern 3: Parent + Related in One Query

```
List<Account> accounts = [
    SELECT Id, Name, Industry, AnnualRevenue,
           (SELECT Id, Name, Email, Phone FROM Contacts),
           (SELECT Id, Name, StageName, Amount FROM Opportunities),
           (SELECT Id, CaseNumber, Status, Subject FROM Cases)
    FROM Account
    WHERE Name LIKE :('%' + accountName + '%')
    LIMIT 1
];

// Access related records
Account acc = accounts[0];
List<Contact> contacts = acc.Contacts;
List<Opportunity> opportunities = acc.Opportunities;
List<Case> cases = acc.Cases;
```

Parameter Sharing Guidelines

When multiple operations use the same parameter (e.g., `accountId`):

Option 1: Each operation accepts accountId

- Best for flexibility
- User can directly query related objects if they know the ID
- Each prompt JSON schema includes `accountId`

Option 2: Some operations find account first

- `find_Account_by_Name` returns accountId
- User then uses that accountId for subsequent queries
- More user-friendly for name-based workflows

Recommended Approach: Support BOTH accountId and accountName in related object operations:

```
{
  "type": "object",
  "required": [],
  "properties": {
    "accountId": {
      "type": "string",
      "description": "ONLY the Account ID. This field is OPTIONAL if accountName is
provided."
    },
    "accountName": {
      "type": "string",
      "description": "ONLY the Account Name. This field is OPTIONAL if accountId is
```

```
  provided."
      }
    }
  }
```

Then in Apex:

```
String accountId = null;

// If accountId provided, use it
if (parameters.containsKey('accountId') &&
    String.isNotBlank(String.valueOf(parameters.get('accountId')))) {
    accountId = String.valueOf(parameters.get('accountId'));
}
// Otherwise, look up by name
else if (parameters.containsKey('accountName') &&
         String.isNotBlank(String.valueOf(parameters.get('accountName')))) {
    String accountName = String.valueOf(parameters.get('accountName'));
    List<Account> accounts = [SELECT Id FROM Account WHERE Name LIKE :('%' +
accountName + '%') LIMIT 1];
    if (accounts.isEmpty()) {
        return errorResponse('Account not found with name: ' + accountName);
    }
    accountId = accounts[0].Id;
}
else {
    return errorResponse('Either accountId or accountName is required.');
}

// Now use accountId for the query
```

File Organization for Multi-Operation Use Cases

Create one folder with multiple prompt JSON files:

```
use-cases/Account_Intelligence/
├── AccountIntelligenceHandler.apex
├── find_Account_by_Name_PromptCommand.json
├── find_Contacts_for_Account_PromptCommand.json
├── find_Opportunities_for_Account_PromptCommand.json
├── find_Cases_for_Account_PromptCommand.json
├── AGENT_DESCRIPTION.txt
└── AGENT_SYSTEM_PROMPT.txt
```

Multi-Operation Agent System Prompt

The system prompt should list ALL available operations:

```
You are an Account Intelligence Agent responsible for finding account information and
related records.
```

```
CORE RESPONSIBILITIES:
- Find accounts by name with fuzzy matching
- Retrieve contacts associated with an account
- Retrieve opportunities associated with an account
- Retrieve cases associated with an account
- Provide comprehensive account intelligence and insights

AVAILABLE OPERATIONS:

1. find_Account_by_Name
   - Finds account records by name
   - Supports partial name matching
   - Returns account details and ID for subsequent queries

2. find_Contacts_for_Account
   - Retrieves all contacts for a specific account
   - Accepts either accountId or accountName

3. find_Opportunities_for_Account
   - Retrieves all opportunities for a specific account
   - Includes stage, amount, and close date information

4. find_Cases_for_Account
   - Retrieves all support cases for a specific account
   - Includes status, priority, and case details

WORKFLOW GUIDANCE:

When users ask about an account:
1. First call find_Account_by_Name if you need the account ID
2. Then use the accountId to call related object operations
3. Alternatively, if user provides account name, you can directly query related objects

EXAMPLE INTERACTIONS:

User: "Find contacts for Acme Corporation"
You: [Call find_Contacts_for_Account with accountName: "Acme Corporation"]
If Success: "Found [X] contacts for Acme Corporation: [list top contacts]"

User: "Show me everything about GenePoint"
You:
Step 1: [Call find_Account_by_Name with accountName: "GenePoint"]
Step 2: [Call find_Contacts_for_Account with accountId from step 1]
Step 3: [Call find_Opportunities_for_Account with accountId from step 1]
Step 4: [Call find_Cases_for_Account with accountId from step 1]
Then provide comprehensive summary of all data.

LIMITATIONS:
You CANNOT:
- Create, update, or delete account records
- Modify contact, opportunity, or case records
- Access records the user doesn't have permission to view

Always work within the user's Salesforce security context.
```

Salesforce Object ID Prefixes

When validating IDs, use these common prefixes:

Object	Prefix
Account	001
Contact	003
Lead	00Q
Opportunity	006
Case	500
Task	00T
Event	00U
Custom Objects	a[0-9][0-9]

Example: Complete Use Case Generation

Input: "Update case status to 'In Progress' or 'Closed' with resolution notes"

Output:

Folder: `use-cases/Update_Case_Status/`

Files:

1. `UpdateCaseStatusAgenticHandler.apex` (175 lines)
2. `update_Case_Status_PromptCommand.json` (30 lines)
3. `AGENT_DESCRIPTION.txt` (3 sentences)
4. `AGENT_SYSTEM_PROMPT.txt` (70 lines)

Naming:

- Prompt Name: `update_Case_Status`
- Handler Class: `UpdateCaseStatusAgenticHandler`
- Method Name: `updateCaseStatus`
- Request Param: `update_Case_Status`

Parameters:

- caseId (string, required) - Salesforce Case ID
- status (string, required, enum) - New status value
- resolutionNotes (string, optional) - Resolution comments

Validations:

- Case ID format (15/18 chars, starts with '500')
- Status value (must be in valid list)
- Case exists
- Case not already closed (if changing to open status)

Response:

- success, status, message, caseId, caseNumber, oldStatus, newStatus, subject, redirectUrl

---

Final Instructions

When you receive a use case:

1. **Analyze** the requirements carefully — what is the agent's domain, objects, operations, and audience?
2. **Extract** operation type, object, required/optional parameters
3. **Apply naming conventions** exactly as specified (folder, handler, prompt name, Ext_ID prefix)
4. **Generate all 5 files** with complete implementation
5. **Design intents** by reading the system prompt and asking: "*What conversation patterns should automatically trigger backend actions?*"
6. **Choose action types** deliberately — at least one Canned Response per intent; add Create Record, Update Field, Flow, or Apex where the situation warrants it
7. **Generate 3 CSV snippet blocks** ready to append to the master files
8. **Validate** against the quality checklist
9. **Output** files in a clear, organized format

Do NOT:

- Skip any sections or files
- Use placeholder comments like "// Add logic here"
- Generate incomplete code
- Forget error handling
- Omit examples in system prompt
- Use HTML tags in system prompt
- Design intents that are too generic (e.g. "user says something bad") — be specific about trigger phrases
- Design all intents with only Canned Response — use backend action types where appropriate
- Use `Invoke Agent` action type — it is not implemented; only use the 5 types: Canned Response, Update Field, Create Record, Flow, Apex

DO:

- Follow every pattern exactly
- Include all validation
- Write production-ready code
- Be specific in all intent trigger descriptions
- Use all 5 action types across the intent set (where scenarios justify them)
- Provide concrete examples in system prompt
- Think through edge cases
- Produce CSV rows that are copy-paste ready to append to master files

---

Created: December 18, 2025 **Version**: 2.0 **Purpose**: Complete guide for LLMs to generate Salesforce AI Agentic Function use cases, including Intent Action Framework design and Master CSV rows